MOUNTX SELF-HOSTING

Phase 2: Parser

422 lines of recursive descent re-expressed as 1386 lines of MountX — producing an AST that dumps byte-identically, node for node, over every file in the repository.

product	selfhost/parser.mx - 1386 lines, 35 mk_* builders
oracle	parser.py - 422 lines, recursive descent
curated gate	49 cases / 496 dump lines - PARITY OK, 16/16 templates
repo gate	38-63 files / 22,265-29,429 dump lines - PARITY OK
census	13,784 nodes / 34 node types / 34 .mx files
divergence	V1 - the port's ladder cliffs at ~250 levels, the oracle's at ~1,000. Loud, accepted, never narrowed away.

*Every figure in this document is taken from the I-M9a brief, from the three design documents in **selfhost/**, or from a measurement taken while this PDF was generated — a line count, a git tag, or a frozen gate artifact. None of it is from memory. The commit range above is* `first-touching-commit..HEAD` *for this phase's own product file, so* `git log` *over it reproduces the phase's history.*

commit range	2e9830ea559d..c293f17
final tag	self-hosted-complete
generated	2026-10-02 against tree c293f17 (2026-10-02)

The tree named above is the tip these figures were measured against — the parent of the commit carrying this PDF. A document cannot record its own hash, so the commit that names it is `i-m9`*, the tag this work is released under.*

2 THE CONTRACT

The governing document is `selfhost/PARSER_DESIGN.md`, written *against* the port and the harness phase 1 delivered — the same three-layer shape (generator, MountX driver, oracle driver, checked-in golden) pointed at a larger surface. It collects eight numbered decisions and one accepted divergence.

The eight decisions (PARSER_DESIGN.md section 18)

#	issue	decision taken	the alternative that lost
P1	node representation	one Node class, kind + sparse fields, 35 mk_* builders	34 subclasses: the same builders, plus a name-to-class table nobody needs
P2	precedence ladder	transcribe the recursion — the ladder survives 250 levels, and the deepest parenthesization in the repo is 2	an explicit-stack rewrite, which is a second parser to keep in sync
P3	number equality in the AST	value equality, coerced with "" — 24 of 24 adversarial texts agree today	compare value_type only for FLOAT and keep a separate value-parity test, if the repo stage ever shows a drift
P4	where token conversion lives	tok_value(tok) in parser.mx	a Token.value_converted() method, which blurs E3 and hands phase 3 a footgun
P5	token spelling in the dump	absent, as in the oracle; alias cases pin it	carry the lexeme on every node — a field the oracle does not have, so it would need inventing
P6	class fields	dump as FIELD, two fields, modelled as a pair	synthesise a Let type for them, which the oracle never emits
P7	dead code	omit peek (zero call sites); keep current's past-end EOF	port both verbatim, or drop both and risk an index error where the oracle has a diagnostic
P8	repo stage scope	.mx only	keep .py, which adds six identical "Program must start with mountx" lines

The parity contract: A1 - A5 (PARSER_DESIGN.md section 3)

	the rule	why
A1	AST equality is a canonical dump, not an object comparison: one line per node, pre-order, |-separated, ending in an EOF line	the AST analogue of E1, and it exists because there is no way to compare a MountX object graph with a Python dict from the outside
A2	diagnostic equality is at the message level — E2 carries over unchanged	parser.py raises SyntaxError, main.py prints it verbatim because the text starts with [, and the port throwxes the identical string. 16 templates, all 16 reached.
A3	numbers in the AST are compared as values, not lexemes — the one place phase 2 deliberately breaks E3	not a contradiction: E3 governs the token stream, where lexer.py keeps raw text. The AST does not — lexer.py has already turned 007 into the int 7 before parser.py sees it, so the oracle has no lexeme left to compare. Value comparison is forced, and measured safe.
A4	both sides parse the same string, wrapper included	load-bearing: the wrapper shifts every line field by one, so if one side wraps and the other does not, the diff is a million lines and meaningless
A5	explicitly out of scope for equality	process exit codes, printx rendering of anything not in the dump, Python repr of a node object, and py_repr above U+00FF (D3/D4 carry over)

The frozen constraints

- **No host edits, no VERSION bump.** Section 20 promised README.md and AGENTS.md would each gain a paragraph. AGENTS.md did — the two parse-parity commands joined the test protocol, with the trigger rule and the divergence list. README.md could not: the file does not exist in this checkout, while AGENTS.md refers to it five times. Nothing was owed there anyway — a reader-facing document should not gain a paragraph about a compiler component no command-line flag can reach.
- **DESIGN.md section 7 still applies in full.** No closures, no chr/ord, quadratic string and array growth, code-point indexing, no negative index. A port that ignored those would be a port of a language that does not exist.
- **One Node class, 35 builders.** Not 34 subclasses, not a class per node type: the builders are the API, and they are the hazard phase 3 inherited (see I-M3's mk_super collision).
- **fbtest.mx is untouched.** No user-observable change, so its header check count did not move.
- **Both wrapper rules are deliberate, and they differ on purpose.** The curated stage *never* wraps, or template 2 (Program must start with mountx) is unreachable by construction. The repo stage wraps by main.py:30-33's predicate, which asks whether the *first token* is MOUNTX — 28 of the 38 files are headerless and several carry a comment before any header, so the naive text test would have produced 28 bogus errors instead of 38 dumps.

3 THE MEASUREMENTS

Every figure here is a gate result, a line count taken from the tree while this PDF was generated, or a number carried verbatim from `selfhost/PARSER_DESIGN.md`.

measure	value	note
Product	`selfhost/parser.mx, 1386 lines, 35 mk_* builders`	plus parse_parity.mx (82) and parse_repo.mx (60) as drivers
Oracle	`parser.py, 422 lines`	one Parser class, one method per construct, plain dicts out. No precedence table, no visitor, no node classes.
Census (measured)	`13,784 nodes / 34 node types / 34 .mx files`	deepest AST nesting 16. The one failure is fbtest.mx, which contains its own mountx and cannot be made a single program by wrapping.
Gate - curated	`49 cases / 496 dump lines — PARITY OK, 16/16 templates reached`	regenerates byte-identically from one dict via gen_pcases.py; the oracle side is generated, the port has to reach the same bytes by its own means
Gate - repo	`38-63 files / 22,265-29,429 dump lines — PARITY OK`	the P-M1 repo stage already walks every .mx, so four milestones were swallowed: P-M2 (parser parses lexer.mx), P-M3, P-M4 and P-M5 (the port parses its own source) were all met inside P-M1. Recorded, not hidden.
Diagnostics	`16/16 templates, each from its own case`	all seven rows of the err_value table have their own case (50-60), rather than being sampled
Port speed	`~57 nodes/s per layer (22,227 nodes / 387 s)`	the number phase 3 designs its entire wall-clock budget against
Recursion probe	`250 levels OK / 275 dies — explicit-stack rewrite rejected`	frames at the boundary: 2,250 fine, 2,475 not, so the host budget is roughly 2,300 MountX frames. The deepest paren nesting in the repository is 2.
Divergence	`V1 — the port's ladder gives out near 250 levels`	the oracle reaches ~1,000 under sys.setrecursionlimit(10000). Loud: maximum recursion depth exceeded, reported as a crash with the runtime line attached, never as a pass.

The two findings that changed the design, and both came from dumping

- **Class fields are not nodes.** parser.py appends a dict with no type key, and 48 of them sit in ClassDef.fields across the repo. A dumper that dispatches on node["type"] dies on the first class in stdlib/data/*.mx, and a port that hands every child to a generic it's-a-Node routine will invent a type the oracle never emitted. So the dump carries a 35th shape spelled FIELD, and the port models a field as a pair rather than as a Let node wearing a hat.
- **All 34 key sets are stable across the whole repo.** For every type the union of observed keys equals the intersection — no type gains or loses a key depending on the file. So a fixed per-type field order is a spec, not a guess.

*The rare types are the real coverage risk, and the census names them: Append 1, Continue 1, Export 1, Loop 5, Read 2, Write 2, Super 2, TryCatch 13, New 33, ClassDef 18 — and **Input 0**, produced by inputx and never used by the repo. Everything under about 20 is thin, so the repo corpus cannot certify those branches and the curated corpus has to. Same lesson as phase 1: a corpus that never emits a template is either covering dead code or missing a bug.*

4 DEFECTS AS LESSONS

The corpus paid for itself **before its first green run**, which is the whole argument for building a corpus rather than reading the grammar aloud in MountX. Three defects, all in parser.mx, none visible in a single-line dump.

what was caught	how	the fix	the lesson
ClassDef emitted init=- unconditionally. A class *with* an init lost the whole FuncDef subtree: every method printed, only the init vanished.	The repo stage, on the stdlib. Silent because init=- is a legal rendering for a class with no init, so the output was well-formed and merely wrong, for most of the stdlib.	The conditional is now there. The golden sample in the design document carries |init=c — and that one character is what caught it.	**A legal rendering of the wrong thing is the hardest kind of bug to see.** The dump's own vocabulary is what hid it; the golden's verbosity is what exposed it.
err_value echoed the lexeme for INT and FLOAT. lexer.py converts the number before the parser sees it, so let 007 = 1 has to read got INT (7) and let 1.500 = 1 has to read got FLOAT (1.5). The port printed 007 and 1.500.	A curated case built specifically for the error path. Silent because it is only observable on that path, and only for a literal whose spelling and value differ.	INT and FLOAT render the converted value. Now: EOF renders None, BOOL renders True/False, STRING renders the decoded string, and everything else renders the lexeme — section 14.2 as written.	**The code had drifted from the document, not the other way round.** The fix was to transcribe the spec into the port and then let the corpus say whether the transcription was right.
The NULL hardcode, in the copy that mattered. err_value returned 'nil' for every NULL token, so let nullx = 1 reported a spelling the program does not contain.	The same defect the phase flagged in tok_value, showing up in the one function whose output the dump can see. STRING next to it rendered the lexeme where the oracle has the decoded value — wrong for "a'b" and for 'a\tb'.	NULL renders in both spellings, STRING renders the decoded value, and the two renderers that share a switch cannot drift apart again.	**The same mistake twice is one mistake plus a missing test.** A defect caught in tok_value should have been searched for by shape, not by name; the corpus found the second copy because the dump can see it.

F1 - a claim in the design document that was simply false

Section 16 asserted that `fbtest.mx` contains an inner `mountx` and therefore answers `Unexpected token MOUNTX ('mountx')` at line 5, which made it a free error-parity case. It does not: fbtest.mx has exactly one mountx, on line 4, and the oracle parses the whole file into a Program with 416 statements. Two consequences, both taken. fbtest.mx is in the repo stage as a **successful** parse — still the most valuable thing it can be here, the largest pure-parse file in the tree and the largest file the two parsers must agree on statement for statement. And the construct that row was invented to cover had no case at all, so it has one: a second mountx on line 3, answering the message and the line the design predicted, now in a file that shows it rather than a sentence that claims it.

V1 - the recursion cliff, and the disposition

Accepted as a host limit. No setrecursionlimit change, no main.py change, no precedence-climbing rewrite. The corpus pins the shape well below the cliff — 120 parens and 10 nested blocks, both matching — and the harness treats a RecursionError as a loud crash rather than a pass. The design's arithmetic: the port's ladder is nine frames a level, the deepest paren nesting in the tree is 2 and the deepest block nest is 4, so the worst file in the worst case needs 2×9 + 4×4 = **34 frames against a budget of about 2,300**. Trading 68× headroom for a second implementation of the grammar is not a deal. And the rule that came out of it is now the project's: **never lift the Python recursion limit to make a parity run pass** — the limit is exactly what makes the cliff loud, and raising it hides the divergence instead of removing it.

5 THE ARTIFACTS

Same three-layer shape as phase 1, one layer larger. The repo stage is where the bugs were; the curated stage is what makes the loop bearable.

```
# stage 1 - the curated corpus, 49 cases from one dict
$ python3 selfhost/parity_parse.py --check
PARITY OK (curated): 49 cases, 496 dump lines, port 8.33s, \
                     16/16 templates reached

# stage 2 - every .mx in the repository
$ python3 selfhost/parity_parse.py --repo --check
PARITY OK (repo): 129 cases, 39561 dump lines, port 848.52s

# the phase's own figures, for the record:
#   curated  49 cases, 496 dump lines, 0 differences
#   repo     38 .mx, 22,265 dump lines, 22,227 nodes, 1.50 MB, 387.03s
#            (measured twice: 414.5 s and 387.0 s)
#   phase 1  PARITY OK: 52 cases, 255 lines  -  still green
# 129 files / 39,561 lines is the tree at report time.
```

Both stages above were re-executed while this PDF was generated. The repo run walked 129 .mx files, dumped 39,561 lines on each side, and compared them byte for byte in 848.52 s of port time. The phase's own 387 s figure is over the 38 files that existed when it was written; the corpus has grown by the self-hosting files twice over since, which is why AGENTS.md says to re-measure before quoting 57 nodes per second at a plan.

What each piece is

file	role
`selfhost/gen_pcases.py`	writes the 49 cases in pcases/ from one dict; the oracle side is generated
`selfhost/pcases/NN-*.txt`	the curated corpus, checked in, regenerating byte-identically
`selfhost/parser.mx`	the port: one Node class, 35 mk_* builders
`selfhost/parse_parity.mx`	MountX driver for the curated corpus (82 lines)
`selfhost/parse_repo.mx`	MountX driver for the repo walk (60 lines)
`selfhost/parity_parse.py`	oracle driver, the --check that diffs the two streams, and the template-coverage audit that reports which case produced which message
`selfhost/parse_repo_files.txt`	the manifest, rebuilt each run

Why the harness has two different wrapper rules

Two wrapper rules in one harness looks like a bug and is the design. The **curated stage never wraps**: a case must be able to have no header, or template 2 (Program must start with mountx) is unreachable by construction and the whole error surface of parse() goes untested. The **repo stage wraps by main.py's predicate**, which asks whether the first *token* is MOUNTX, not whether the text starts with the word. 28 of the 38 files are headerless and several carry a comment before any header, so the naive text test would have produced 28 bogus Program must start with mountx lines instead of 38 dumps. Porting the predicate exactly is what made the repo stage meaningful on the first try.

What phase 3 inherits

- **~57 nodes/s per layer.** An interpreter layer over the same 22,227 nodes is not free, and fbtest.mx as the differential anchor runs 277 assertions through three layers if the port drives it. Budget the wall time before designing the corpus, not after.
- **One host stack, two spenders.** The ~2,300-frame budget is consumed by the port's own recursion *and* by every guest call it makes through it.
- **The three-command pattern works, and is now the house shape.** Generator, MountX driver, oracle driver, golden corpus checked in as files, plus a repo stage. Phase 3 was told not to invent a fourth shape.
- **A corpus finds what a reading of the grammar cannot.** The interpreter port is a much larger body of logic — natives, the environment chain, values_equal, super — and the odds that a transcription of it is right first time are correspondingly smaller. The differential harness is the deliverable, not the .mx file.

6 THE CHAIN

12 tags carry the self-hosting work, in the order they were made. The list below is read from `git for-each-ref --sort=creatordate refs/tags` at generation time, with each tag peeled to the commit it points at, so the hashes are the repository's and not this document's.

#	tag	date	commit	what it froze
1	`i-m0-baseline`	2026-10-01	`2e9830ea559d`	MountX I-M0 baseline: interp.mx value layer (10 tags, display, identity stamps, kind slot) certified by 5-gate differential. Lexer 59/46,944 + parser 49/26,501 byte-exact, fbtest 277, 67/67 oracle pins, 13-mutation control battery.
2	`i-m2-shipped`	2026-10-01	`1785ee96ce67`	I-M2 shipped: calls, control flow, value layer, natives; 8-guest exec gate green
3	`i-m2-review`	2026-10-01	`9a97b5453ccc`	MountX I-M2 review (Q1): MAX_GUEST_DEPTH 100 -> 40, because 100 was inert too
4	`i-m3-errors`	2026-10-01	`032342a92e37`	MountX I-M3: the error pass - 30-program message sweep, mk_super collision fix, D-1 search half
5	`i-m4`	2026-10-01	`469b167c4e02`	MountX I-M4: the class layer - ClassDef, New, instance/super DotAccess, Input
6	`i-m5`	2026-10-01	`4cbf5651ba1f`	MountX I-M5: the closure pass - loops proven, sweep measured, not counted
7	`i-m6-prep`	2026-10-01	`e882d58e0b97`	I-M6 prep: close I16/I17/I10, and fix a message the audit could not see
8	`i-m6`	2026-10-01	`0ac2d56830b4`	I-M6: the anchor - interp.mx runs fbtest.mx byte-for-byte
9	`i-m7`	2026-10-02	`373d319a960e`	I-M7: the layer boundary - a delegated native answers the HOST's context
10	`i-m7.1`	2026-10-02	`68528f66f9de`	I-M7.1: the crown runs - and the second defect class, an ORDER not a transcription
11	`self-hosted-complete`	2026-10-02	`68528f66f9de`	I-M7.1: the crown runs - and the second defect class, an ORDER not a transcription
12	`i-m9-docs`	2026-10-02	`40d0b5cc6a19`	I-M9: the MountX documentary, and a gate that keeps it true

Hashes are the first 12 characters of the object each tag peels to. `git rev-parse <tag>` prints the untruncated 40, so any row can be checked against the repository rather than against this document.

Where this phase sits

- **Phase 2** is frozen at the head of the chain, and `self-hosted-complete` carries the phase's claim in full.
- **Phase 1 (lexer)** and **Phase 2 (parser)** have no tags of their own. They are proven continuously by their gates, and their products — `lexer.mx` (731 lines) and `parser.mx` (1386 lines) — are unchanged across every tag in the list.
- **Phase 3 (interpreter)** owns every tag from `i-m0-baseline` onward. Of the 12 tags in the chain, 12 are phase-3 work; the newest is this documentary, released as `i-m9`.

What the chain asserts

Phase 2's claim: **for every program, the MountX parser produces an AST that dumps identically to parser.py's** — proved on 49 curated cases covering all sixteen diagnostic templates and on every .mx file in the repository, with the port parsing its own source as one of the cases. The accepted divergence is a recursion cliff that is loud, bounded, and recorded rather than engineered away.

```
$ git --no-pager for-each-ref --sort=creatordate \
      --format='%(refname:short) %(creatordate:short) %(objectname)' \
      refs/tags
  # run at generation time; this is what the table above is
  # (add | cut -c1-12 to print hashes untruncated)
```

MountX I-M9a — documentary phase reports. Generated by `docs/make_phase_pdfs.py` on 2026-10-02 at 13:02, against tree c293f17, for tag `i-m9`.